Hate Speech Online and the Role of NLP Toolkits in Combating It

Introduction

After studying the rapid growth of harmful discourse on social media platforms, researchers have found that the most effective responses combine the analytical power of Natural Language Processing (NLP) with a human-centred understanding of why hate speech causes real harm.

By documenting the current state of the problem and revealing a path toward automated, scalable detection, this report highlights the gap between current moderation systems and the future potential of AI-driven moderation.

This report follows a three-part structure:

1. Understanding the current problem
2. Exploring NLP-based solutions
3. The End – Presenting the future vision and call to action

Understanding the current problem

What Is: The Scale of Online Hate Speech

Online hate speech has increased significantly across social media platforms. Harmful language spreads rapidly due to the scale and speed of user-generated content, making manual moderation difficult and inefficient.

To build automated detection systems, researchers require large-scale labelled datasets containing real-world social media interactions.

The Dataset: A Window Into Online Sentiment

The Sentiment140 dataset, derived from the Twitter API, provides a large collection of labelled tweets for sentiment analysis and NLP experimentation.

The dataset is stored in CSV format and contains tweets with emoticons removed.

Dataset Fields

Field	Name	Example	Description
0	Polarity	0 / 2 / 4	Sentiment label: 0 = Negative, 2 = Neutral, 4 = Positive
1	Tweet ID	2087	Unique identifier of the tweet
2	Date	Sat May 16 23:58:44 UTC 2009	Timestamp of tweet posting
3	Query	lyx / NO_QUERY	Search query used
4	Username	robotickilldozr	Twitter username
5	Tweet Text	Lyx is cool	Raw tweet content

The dataset primarily uses a binary sentiment scale where:

- `0` → Negative sentiment
- `4` → Positive sentiment

The tweet text field serves as the primary analytical input for NLP models.

Although Sentiment140 was designed for sentiment analysis, negative tweets may contain abusive or toxic language, making the dataset useful for hate speech research.

Exploring NLP-based solutions

Current Problems vs Future Possibilities

What Is

Platforms currently depend on manually created keyword blocklists that:

- Are slow to update
- Can be bypassed easily
- Fail to understand sarcasm or context

What Could Be

Modern NLP classifiers trained on large datasets can:

- Detect hate speech with very high accuracy
- Process millions of posts quickly
- Continuously adapt to evolving language

Human Moderation Challenges

What Is

Human moderators experience:

- Psychological stress
- High burnout rates
- Inconsistent decision-making

What Could Be

An NLP-powered moderation layer could:

- Automatically filter obvious violations
- Send only complex cases to human reviewers
- Reduce moderator workload and mental strain

Transforming Raw Data into Intelligence

What Is

The Sentiment Analysis dataset is simply a raw CSV file.

What Could Be

Using NLP preprocessing techniques, the dataset can become a powerful supervised learning corpus for training hate speech classifiers.

Core NLP Techniques for Hate Speech Detection

Modern NLP systems combine multiple approaches for accurate moderation.

---

1. TF-IDF and Bag-of-Words

These techniques identify:

- Frequently used offensive terms
- Harmful word patterns
- N-grams linked to abusive language

They provide fast and efficient baseline models.

---

2. Word Embeddings

Models such as:

- Word2Vec
- GloVe

capture semantic similarity between words, enabling systems to identify harmful expressions even when wording changes.

---

3. Transformer Models

Advanced transformer models like:

- BERT
- RoBERTa

understand full sentence context and detect:

- Sarcasm
- Implicit hate
- Irony
- Coded language

---

4. Named Entity Recognition (NER)

NER identifies:

- Individuals
- Communities
- Organisations

being targeted in hateful content.

---

5. Ensemble Methods

Ensemble learning combines multiple classifiers to:

- Reduce false positives
- Improve recall
- Balance moderation accuracy

Mapping the Dataset to the NLP Pipeline

Dataset Field	NLP Pipeline Usage
Polarity	Target label
Tweet Text	Input feature
Query	Domain tag
Date	Temporal analysis

Text Preprocessing Steps

The tweet text undergoes several preprocessing operations:

1. Unicode normalization
2. Lowercasing
3. URL removal
4. Mention removal

5. Tokenisation
6. Stopword removal
7. Stemming
8. Lemmatization

After preprocessing:

- TF-IDF converts text into numerical vectors
- Embeddings convert text into dense semantic representations

---

Proposed System and Methodology

The project implementation follows a standard NLP pipeline.

1. Data Collection

Tweets are collected from the Sentiment140 dataset.

2. Data Cleaning

Removal of:

- URLs
- Mentions
- Emojis
- Punctuation

3. Tokenisation

Text is split into meaningful tokens using NLTK.

4. Stopword Removal

Common non-informative words are removed.

5. Stemming and Lemmatization

Words are reduced to root forms.

6. Feature Extraction

TF-IDF vectorisation converts text into machine-readable vectors.

7. Model Training

Algorithms used include:

- Naive Bayes
- Logistic Regression

8. Sentiment Prediction

The trained model predicts sentiment polarity.

9. Performance Evaluation

Evaluation metrics include:

- Accuracy

- Precision
- Recall
- F1-Score
- Confusion Matrix

Tools and Technologies

- Python
- NLTK
- Pandas
- NumPy
- Scikit-learn
- Matplotlib
- Seaborn
- Jupyter Notebook

Ethical Considerations and Limitations

Automated hate speech detection systems may produce:

- False positives
- False negatives

due to:

- Sarcasm
- Cultural variation
- Contextual ambiguity
- Evolving language patterns

Therefore, human oversight remains necessary to ensure:

- Fairness
- Transparency
- Responsible moderation

Presenting the future vision and call to action

Call to Action

Researchers, platform operators, and policymakers must work together to operationalise NLP-based moderation systems at scale.

The tools already exist:

- Large datasets
- Open-source NLP libraries
- Proven machine learning methods

The challenge is no longer technical — it is organisational.

A practical first step is deploying a fine-tuned BERT classifier on high-volume content streams and evaluating its performance against human-reviewed data.

New Bliss: The Future of Safer Digital Spaces

Imagine a digital environment where harmful content is detected within seconds instead of hours or days.

Imagine a system where:

- Human moderators focus only on nuanced cases

- Marginalised communities participate safely online
- Platforms respond at the same scale as the threat itself

That is the promise of NLP-powered moderation.

While no technology can eliminate online hate completely, NLP fundamentally changes the balance — enabling platforms to move from reactive moderation to proactive protection.

Conclusion

Natural Language Processing provides a scalable and intelligent solution for detecting hate speech online.

By combining:

- Large-scale datasets
- Advanced machine learning models
- Human oversight

platforms can create safer and more inclusive digital communities.

The future of online moderation lies not in replacing humans, but in empowering them with intelligent NLP systems capable of operating at internet scale.

References

1. NLTK Documentation - https://www.nltk.org/
2. Scikit-learn Documentation - https://scikit-learn.org/
3. Sentiment Analysis Dataset Source
4. Python Official Documentation - https://docs.python.org/3/

Twitter Sentiment Analysis using NLTK

Exploratory Data Analysis (EDA)

Prepared by: Ballem Dileep

Introduction

This project focuses on analyzing Twitter sentiment data using Natural Language Processing techniques. The objective is to understand the dataset structure, perform exploratory data analysis, and prepare the data for sentiment classification tasks.

Problem Statement

The objective of this project is to analyze Twitter data and classify tweets into positive and negative sentiments using Natural Language Processing (NLP) and Machine Learning techniques. The project utilizes NLTK for text preprocessing and multiple machine learning models for sentiment classification.

In [1]:
```
# Step 1: Import Required Libraries
# In this step, all necessary Python libraries for data analysis and visualization are imported.

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import seaborn as sns

# Optional
import warnings
warnings.filterwarnings('ignore')

# Style
sns.set_style("whitegrid")
```

In [2]:
```
# Step 2: Load the Dataset
# The Sentiment140 dataset is loaded into the notebook using Pandas.

df = pd.read_csv(r"C:\Users\balle\OneDrive\Desktop\project O\ds project\Dataset_Sentiment Analysis of Twitter using NL toolkit
```

In [3]:
```
# Step 3: Add Column Names
# Meaningful column names are assigned to improve readability and analysis.

df.columns = ['target', 'id', 'date', 'flag', 'user', 'text']
```

In [4]:
```
# Step 4: Check Dataset Shape
# The number of rows and columns in the dataset is identified.

print("Rows and Columns:", df.shape)
```

```
Rows and Columns: (1599999, 6)
```

In [5]:
```
# Step 5: Display First Five Rows
# The first few records of the dataset are displayed to understand the structure and content.

df.dtypes
```

Out[5]:
```
target     int64
id         int64
date      object
flag      object
user      object
text      object
dtype: object
```

In [6]:
```
# Step 6: Check Data Types
# The datatype of each column is analyzed to understand the dataset schema.

df.isnull().sum()
```

Out[6]:
```
target    0
id        0
date      0
flag      0
user      0
text      0
dtype: int64
```

In [7]:
```
# Step 7: Check Missing Values
# This step identifies whether any null or missing values exist in the dataset.

missing_percentage = (df.isnull().sum() / len(df)) * 100
print(missing_percentage)
```

```
target    0.0
id        0.0
date      0.0
flag      0.0
user      0.0
text      0.0
dtype: float64
```

In [8]:
```
# Step 8: Calculate Missing Value Percentage
# The percentage of missing values in each column is calculated.

df.duplicated().sum()
```

Out[8]:
```
np.int64(0)
```

In [9]:
```
# Step 9: Check Duplicate Records
# Duplicate rows in the dataset are identified.

df = df.drop_duplicates()
```

In [10]:
```
# Step 10: Remove Duplicate Values
# Duplicate records are removed to improve data quality.

df.describe()
```

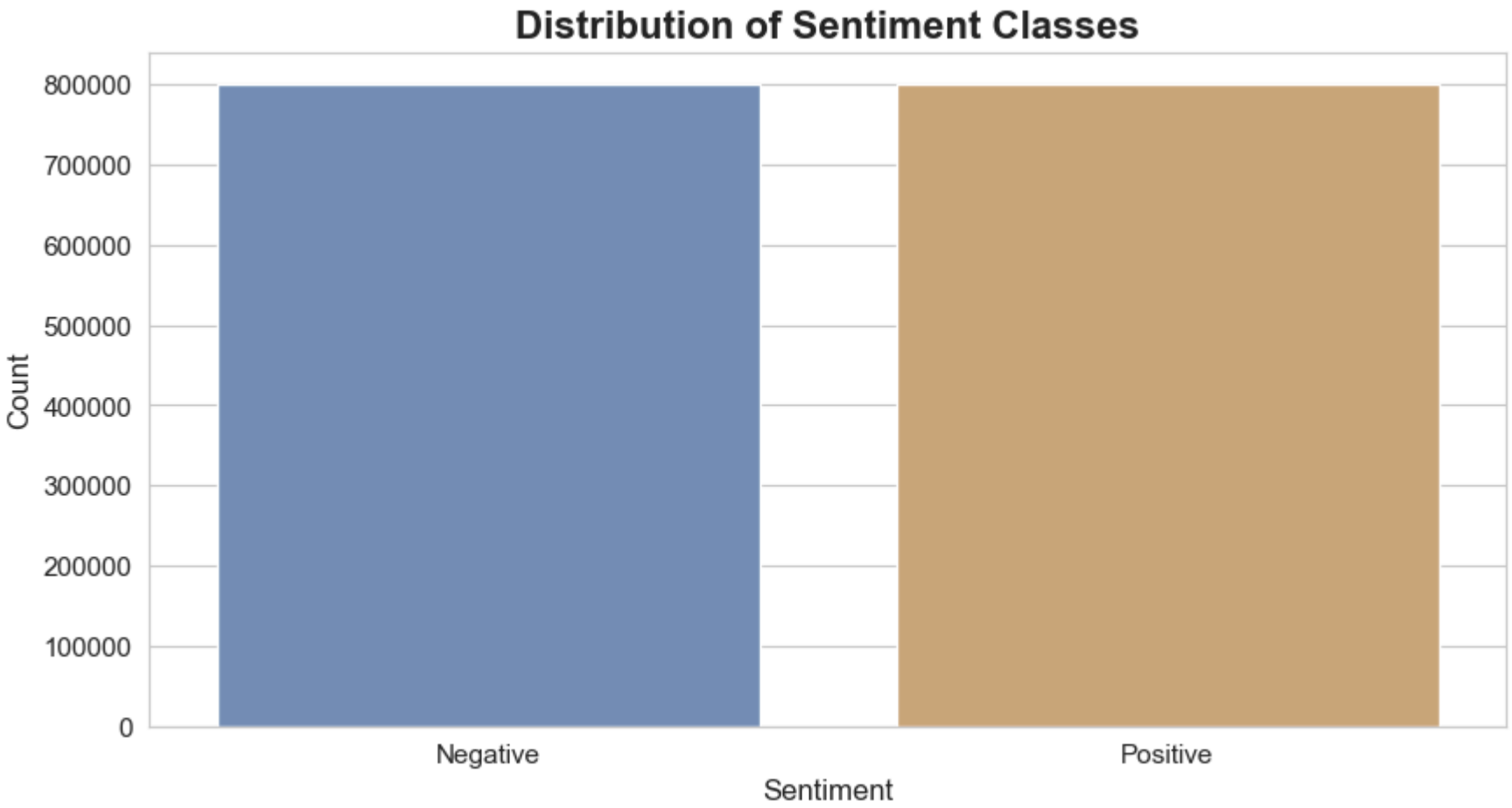
Out[10]:

	target	id
count	1.599999e+06	1.599999e+06
mean	2.000001e+00	1.998818e+09
std	2.000001e+00	1.935757e+08
min	0.000000e+00	1.467811e+09
25%	0.000000e+00	1.956916e+09
50%	4.000000e+00	2.002102e+09
75%	4.000000e+00	2.177059e+09
max	4.000000e+00	2.329206e+09

In [11]:
```
df['sentiment'] = df['target'].replace({0:'Negative', 4:'Positive'})
```

In [12]:
```
# Step 11: Visualize Target Variable Distribution
# The distribution of sentiment classes is visualized using Seaborn and Matplotlib.

import matplotlib.pyplot as plt
import seaborn as sns

# Apply Professional Visualization Style
# Pastel color palettes and clean chart formatting are applied to improve visualization quality.
# Professional theme
sns.set_style("whitegrid")

# Professional muted colors
colors = ["#6C8EBF", "#D9A66B"]

plt.figure(figsize=(10,5))
```

```
sns.countplot(
    x='sentiment',
    data=df,
    palette=colors
)

plt.title(
    "Distribution of Sentiment Classes",
    fontsize=16,
    fontweight='bold'
)

plt.xlabel("Sentiment", fontsize=12)
plt.ylabel("Count", fontsize=12)

plt.xticks(fontsize=11)
plt.yticks(fontsize=11)

plt.show()
```



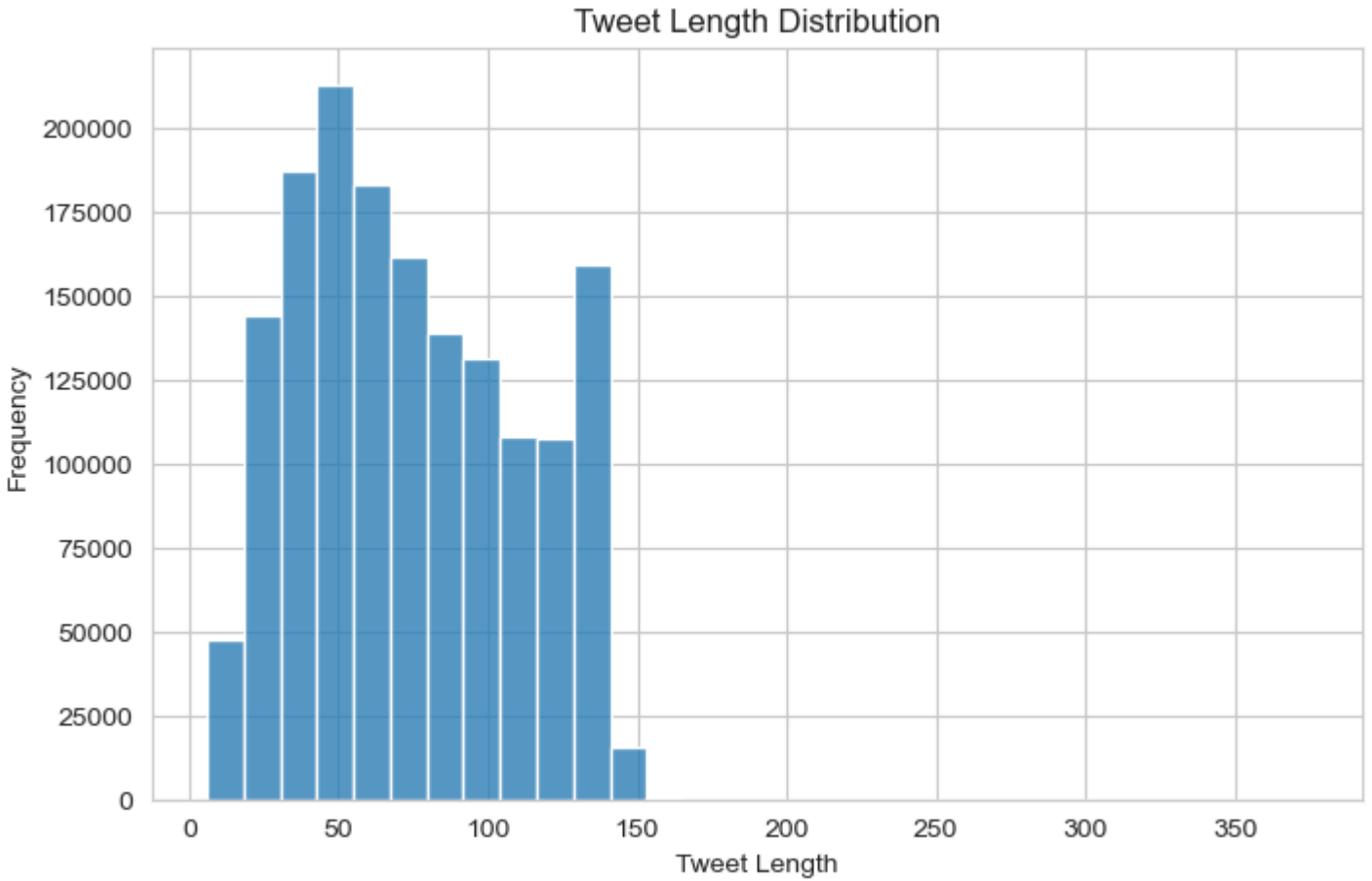
Insight

The dataset contains balanced positive and negative sentiment classes, making it suitable for machine learning classification tasks.

In [13]:
```
df['word_count'] = df['text'].apply(lambda x: len(str(x).split()))
```

In [14]:
```
# Step 12:Tweet Length Distribution
# This visualization shows the distribution of tweet lengths based on the number of characters present in each tweet.
```

```
df['tweet_length'] = df['text'].apply(len)

plt.figure(figsize=(8,5))

sns.histplot(df['tweet_length'], bins=30)

plt.title("Tweet Length Distribution")
plt.xlabel("Tweet Length")
plt.ylabel("Frequency")

plt.show()
```

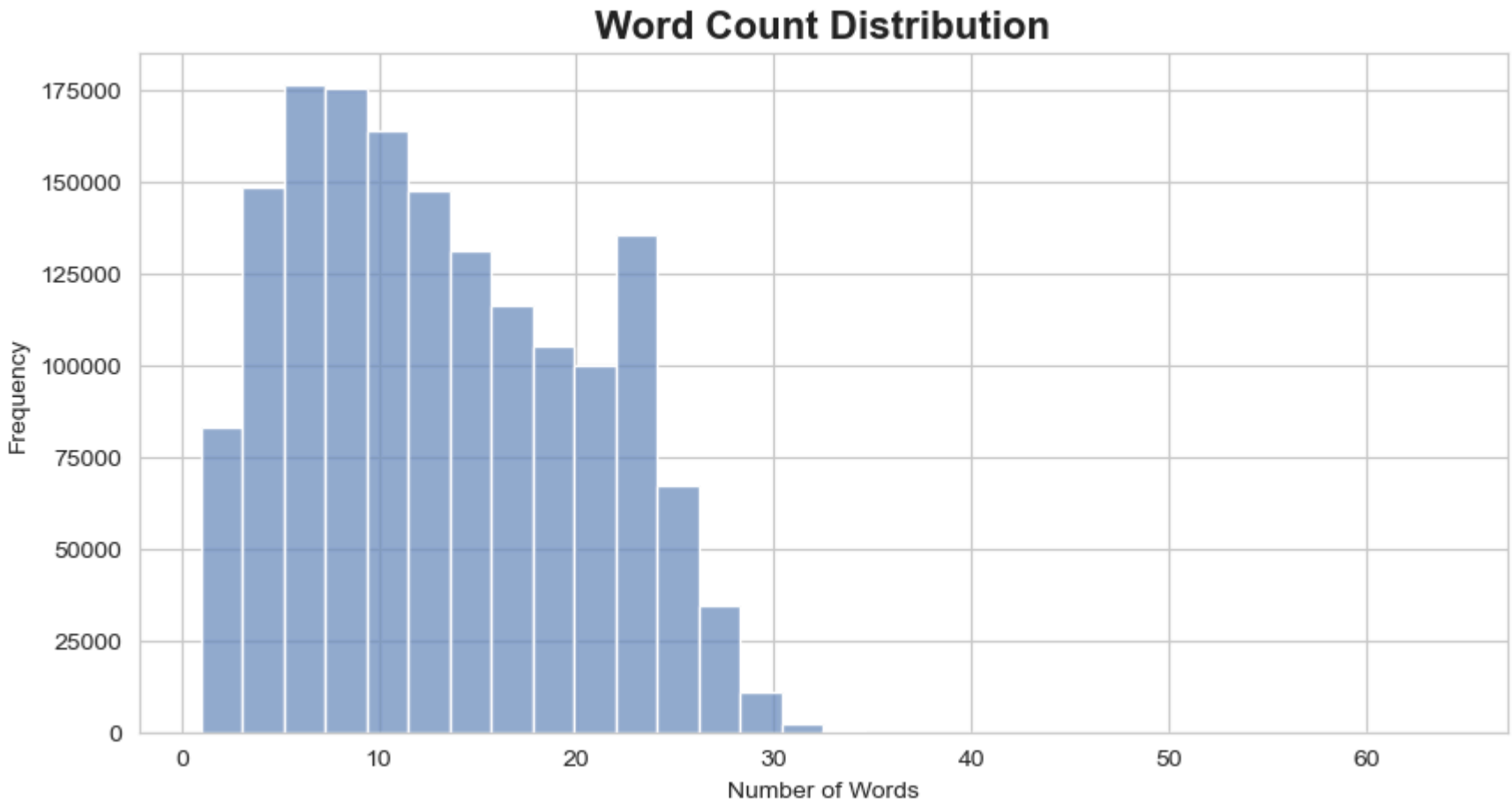


Insight

Most tweets contain a moderate number of characters, which reflects the short-text nature of Twitter data. Tweet length analysis helps understand text variability before preprocessing and model training.

In [15]:
```
df['word_count'] = df['text'].apply(lambda x: len(str(x).split()))
```

In [16]:
```
# Step 13: Word Count Distribution
# This visualization represents the distribution of the number of words used in tweets across the dataset.


plt.figure(figsize=(10,5))

sns.histplot(
    df['word_count'],
    bins=30,
    color="#6C8EBF"
)

plt.title(
    "Word Count Distribution",
    fontsize=16,
    fontweight='bold'
)

plt.xlabel("Number of Words")
plt.ylabel("Frequency")

plt.show()
```



Insight

The majority of tweets contain a relatively small number of words, which is common in social media platforms. Word count analysis is useful for understanding textual complexity and preparing the data for NLP preprocessing techniques.

Conclusion

The Sentiment dataset contains approximately 1.6 million Twitter records with balanced sentiment classes. Exploratory Data Analysis revealed no missing or duplicate values, making the dataset suitable for Natural Language Processing and machine learning tasks. Visual analysis of sentiment distribution and common words provides meaningful insights into positive and negative tweet patterns.

Data Transformation and Model Building

Data Preprocessing

This phase focuses on cleaning and transforming tweet text data using Natural Language Toolkit (NLTK) techniques. The cleaned data is then prepared for machine learning model training and sentiment classification.

In [17]:
```
# STEP 1 – Import Required NLP Libraries

import re
import nltk
import string
import pandas as pd
import numpy as np

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings('ignore')
```

In [18]:
```
# STEP 2 – Download NLTK Resources

nltk.download('stopwords')
nltk.download('punkt')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\balle\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\balle\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\balle\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

Out[18]: True

In [19]:
```
import nltk

nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\balle\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

Out[19]: True

In [20]:
```
# STEP 3 – Convert Sentiment Labels

df['target'] = df['target'].replace(4,1)
```

In [21]:
```
# STEP 4 – Create Stopwords, Stemmer, Lemmatizer

stop_words = set(stopwords.words('english'))

stemmer = PorterStemmer()

lemmatizer = WordNetLemmatizer()
```

In [22]:
```
# STEP 5 — TEXT CLEANING FUNCTION

def clean_text(text):

    # Convert to lowercase
    text = text.lower()

    # Remove URLs
    text = re.sub(r'http\S+|www\S+', '', text)

    # Remove mentions and hashtags
    text = re.sub(r'@\w+|#', '', text)

    # Remove punctuation
    text = text.translate(str.maketrans('', '', string.punctuation))

    # Remove numbers
    text = re.sub(r'\d+', '', text)

    # Tokenization
    tokens = word_tokenize(text)

    # Remove stopwords
    tokens = [word for word in tokens if word not in stop_words]

    # Stemming
    tokens = [stemmer.stem(word) for word in tokens]

    # Lemmatization
    tokens = [lemmatizer.lemmatize(word) for word in tokens]

    return " ".join(tokens)
```

In [23]:
```
# STEP 6 — Apply Cleaning

df_sample = df.sample(50000, random_state=42)
```

In [24]:
```
df_sample['clean_text'] = df_sample['text'].apply(clean_text)
```

In [25]:
```
# STEP 7 — Show Cleaned Output
df_sample[['text', 'clean_text']].head()
```

Out[25]:

	text	clean_text
541200	@Nkluvr4eva My poor little dumpling In Holmde...	poor littl dumpl holmdel vid realli tryinghop ...
750	I'm off too bed. I gotta wake up hella early t...	im bed got ta wake hella earli tomorrow morn
766711	I havent been able to listen to it yet My spe...	havent abl listen yet speaker bust
285055	now remembers why solving a relatively big equ...	rememb solv rel big equat two unknown total pa...
705995	Ate too much, feel sick	ate much feel sick

Insight

Text preprocessing improves data quality by removing unnecessary symbols, stopwords, URLs, and punctuation. Tokenization, stemming, and lemmatization help standardize textual information for machine learning models.

In [26]:
```
# STEP 8 — Train-Test Split

X = df_sample['clean_text']

y = df_sample['target']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Train-Test Split

The dataset is divided into training and testing sets using an 80-20 ratio. The training dataset is used for model learning, while the testing dataset is used for evaluating model performance.

TF-IDF Vectorization

TF-IDF (Term Frequency-Inverse Document Frequency) converts textual tweet data into numerical feature vectors that can be used by machine learning algorithms for sentiment classification.

In [27]:
```
# STEP 9 — TF-IDF Vectorization

tfidf = TfidfVectorizer(max_features=5000)

X_train_tfidf = tfidf.fit_transform(X_train)

X_test_tfidf = tfidf.transform(X_test)
```

In [28]:
```
# STEP 10 — Bernoulli Naive Bayes

from sklearn.naive_bayes import BernoulliNB
from sklearn.metrics import accuracy_score
```

In [29]:
```
#Train Model
bnb_model = BernoulliNB()

bnb_model.fit(X_train_tfidf, y_train)
```

Out[29]:
```
▾ BernoulliNB
▶ Parameters
```

In [30]:
```
# Predictions

bnb_pred = bnb_model.predict(X_test_tfidf)
```

In [31]:
```
# Accuracy

bnb_accuracy = accuracy_score(y_test, bnb_pred)

print("Bernoulli Naive Bayes Accuracy:", bnb_accuracy)
```

```
Bernoulli Naive Bayes Accuracy: 0.7454
```

Insight

Bernoulli Naive Bayes performs efficiently on binary and sparse textual features generated through TF-IDF vectorization. The model provides a strong baseline for sentiment classification tasks.

In [32]:
```
# STEP 11 – Logistic Regression

from sklearn.linear_model import LogisticRegression
```

In [33]:
```
# Train

lr_model = LogisticRegression()
lr_model.fit(X_train_tfidf, y_train)
```

Out[33]:

▾ LogisticRegression

▶ Parameters

In [34]:
```
# Predict

lr_pred = lr_model.predict(X_test_tfidf)
```

In [35]:
```
# Accuracy

lr_accuracy = accuracy_score(y_test, lr_pred)
print("Logistic Regression Accuracy:", lr_accuracy)
```

```
Logistic Regression Accuracy: 0.7533
```

Insight

Logistic Regression demonstrates strong performance for sentiment classification by effectively separating positive and negative tweet patterns within the TF-IDF feature space.

In [36]:
```
# STEP 12 – Support Vector Machine (SVM)

from sklearn.svm import LinearSVC
```

In [37]:
```
# Train

svm_model = LinearSVC()
svm_model.fit(X_train_tfidf, y_train)
```

Out[37]:
```
▾ LinearSVC
▶ Parameters
```

In [38]:
```
# Predict

svm_pred = svm_model.predict(X_test_tfidf)
```

In [39]:
```
# Accuracy

svm_accuracy = accuracy_score(y_test, svm_pred)
print("SVM Accuracy:", svm_accuracy)
```

```
SVM Accuracy: 0.7422
```

Insight

Support Vector Machines are highly effective for text classification tasks because they can efficiently separate high-dimensional textual data generated by TF-IDF vectorization.

Positive Tweet WordCloud

This WordCloud visualizes the most frequently occurring words in positive tweets after text preprocessing using NLTK techniques.

In [40]:
```
!pip install wordcloud
```

```
Requirement already satisfied: wordcloud in c:\users\balle\anaconda3\lib\site-packages (1.9.6)
Requirement already satisfied: numpy>=1.19.3 in c:\users\balle\anaconda3\lib\site-packages (from wordcloud) (2.3.5)
Requirement already satisfied: pillow in c:\users\balle\anaconda3\lib\site-packages (from wordcloud) (12.0.0)
Requirement already satisfied: matplotlib in c:\users\balle\anaconda3\lib\site-packages (from wordcloud) (3.10.6)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\balle\anaconda3\lib\site-packages (from matplotlib->wordcloud) (1.
3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\balle\anaconda3\lib\site-packages (from matplotlib->wordcloud) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\balle\anaconda3\lib\site-packages (from matplotlib->wordcloud) (4.
60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\balle\anaconda3\lib\site-packages (from matplotlib->wordcloud) (1.
4.9)
Requirement already satisfied: packaging>=20.0 in c:\users\balle\anaconda3\lib\site-packages (from matplotlib->wordcloud) (25.
0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\balle\anaconda3\lib\site-packages (from matplotlib->wordcloud) (3.
2.5)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\balle\anaconda3\lib\site-packages (from matplotlib->wordcloud)
(2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\balle\anaconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib->w
ordcloud) (1.17.0)
```

In [41]:
```python
from wordcloud import WordCloud
```

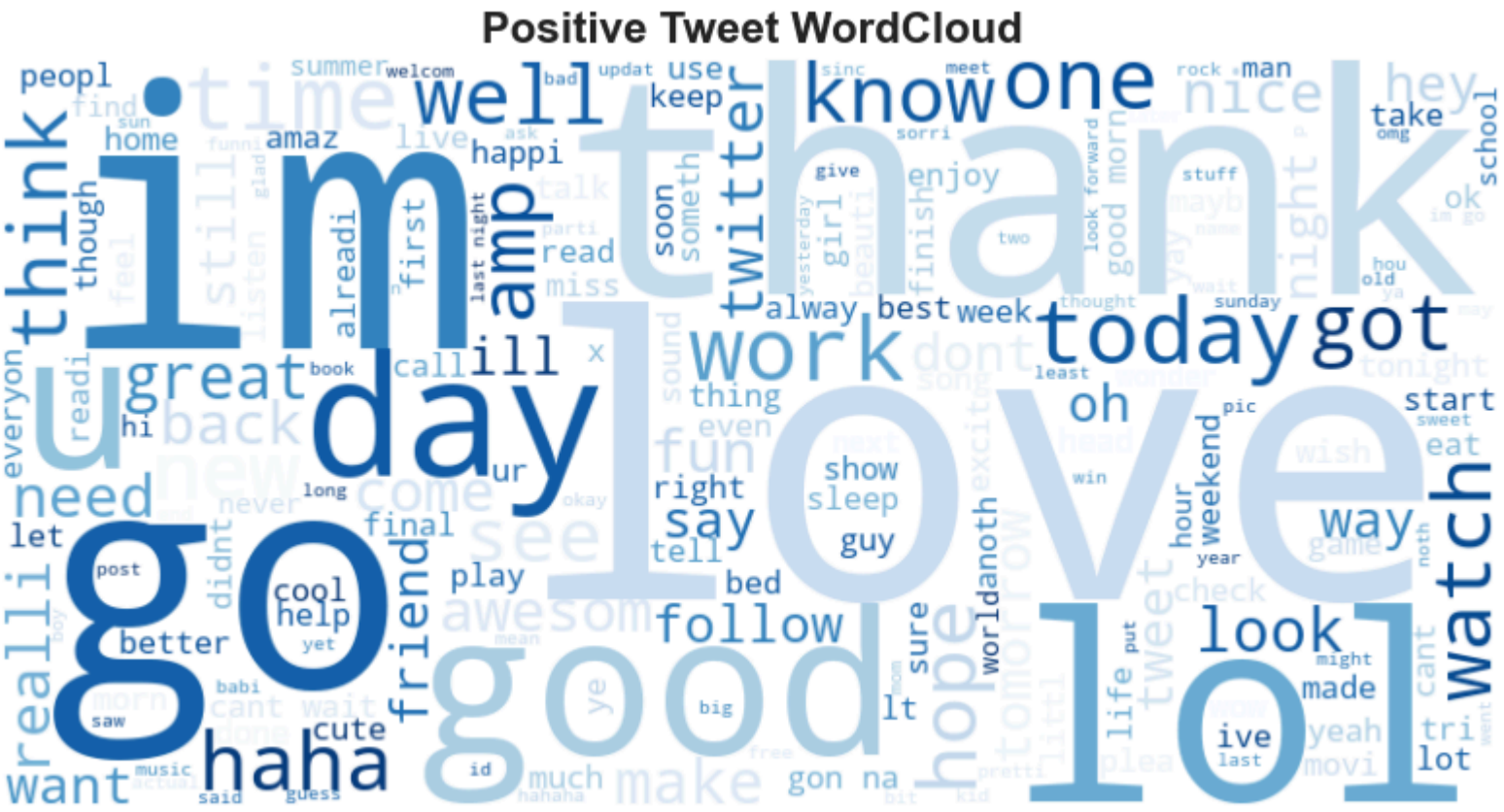
In [42]:
```python
# Create Sample Dataset
df_sample = df.sample(50000, random_state=42)
```

In [43]:
```python
# Apply Cleaning Function Again
df_sample['clean_text'] = df_sample['text'].apply(clean_text)
```

In [44]:
```python
# Separate Positive & Negative Tweets

positive_tweets = df_sample[df_sample['target'] == 1]['clean_text']
negative_tweets = df_sample[df_sample['target'] == 0]['clean_text']
```

In [45]:
```python
# Combine Text

positive_text = " ".join(positive_tweets.astype(str))
negative_text = " ".join(negative_tweets.astype(str))
```

In [46]:
```
# STEP 13 - Positive Tweet WordCloud

positive_wordcloud = WordCloud(
    width=800,
    height=400,
    background_color='white',
    colormap='Blues'
).generate(positive_text)

plt.figure(figsize=(10,5))

plt.imshow(positive_wordcloud, interpolation='bilinear')

plt.axis('off')

plt.title(
    "Positive Tweet WordCloud",
    fontsize=16,
    fontweight='bold'
)

plt.show()
```



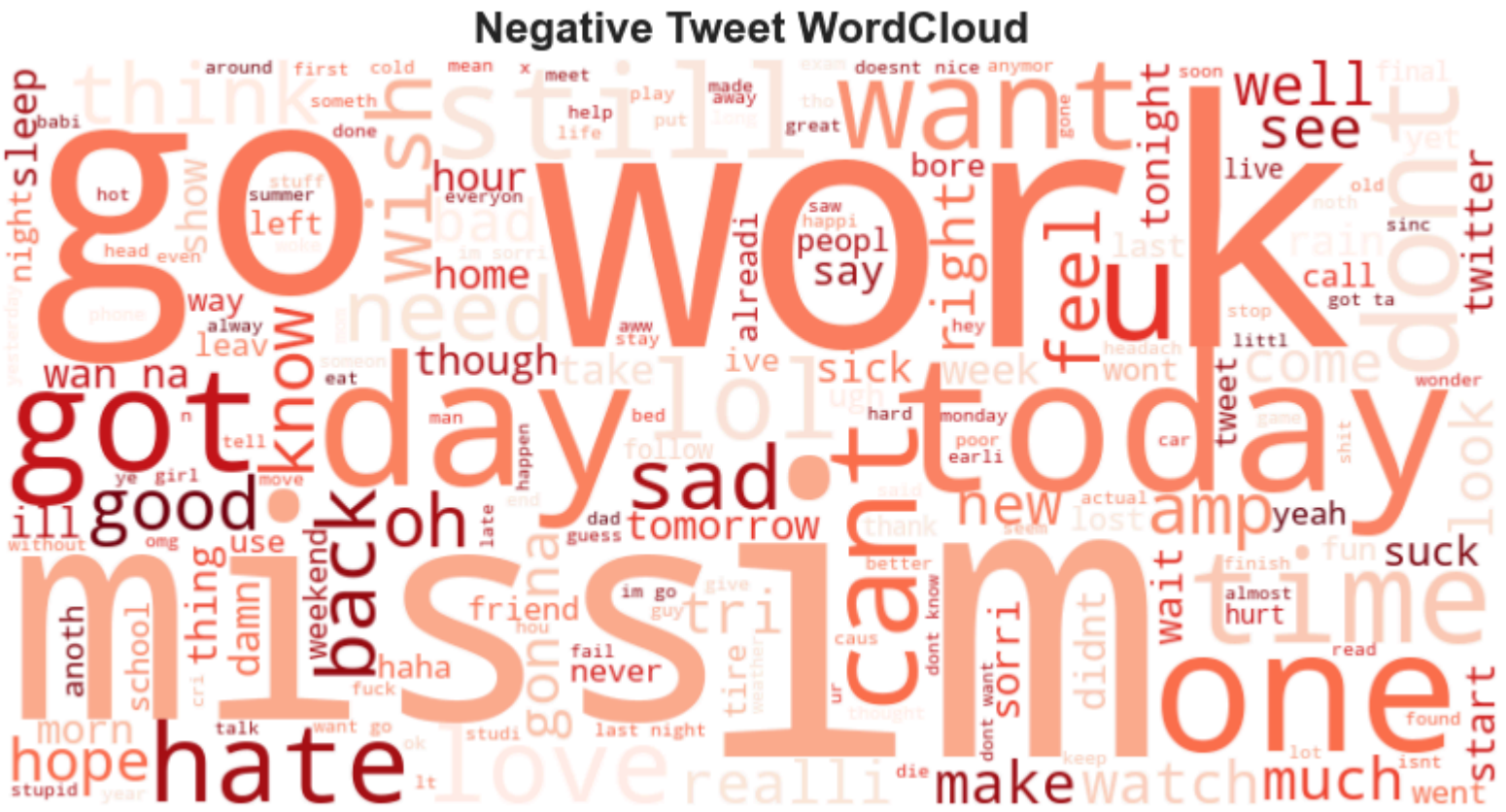
Insight

The WordCloud highlights the most commonly used terms in positive tweets, helping identify patterns associated with positive sentiment and user opinions.

Negative Tweet WordCloud

This WordCloud visualizes the most frequently occurring words in negative tweets after text preprocessing using NLTK techniques.

In [47]:
```
# STEP 14 – Negative WordCloud

negative_wordcloud = WordCloud(
    width=800,
    height=400,
    background_color='white',
    colormap='Reds'
).generate(negative_text)

plt.figure(figsize=(10,5))

plt.imshow(negative_wordcloud, interpolation='bilinear')

plt.axis('off')

plt.title(
    "Negative Tweet WordCloud",
    fontsize=16,
    fontweight='bold'
)

plt.show()
```



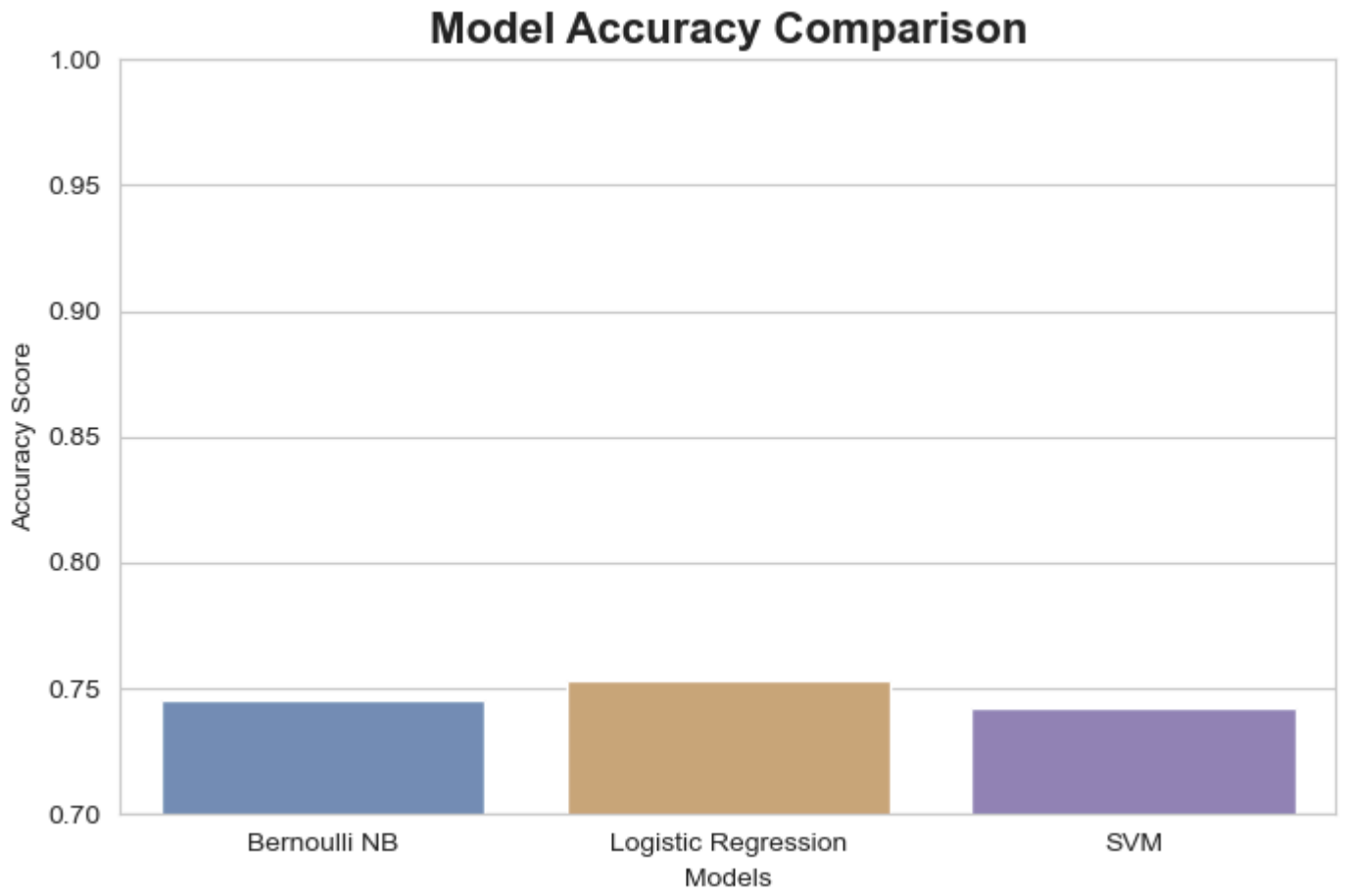
Insight

The negative tweet WordCloud highlights commonly used words associated with negative emotions, complaints, and unfavorable opinions expressed on Twitter.

Model Accuracy Comparison

The performance of Bernoulli Naive Bayes, Logistic Regression, and Support Vector Machine models is compared using accuracy scores on the testing dataset.

In [48]:
```
# STEP 15 — Accuracy Comparison

models = ['Bernoulli NB', 'Logistic Regression', 'SVM']

accuracies = [
    bnb_accuracy,
    lr_accuracy,
    svm_accuracy
]

plt.figure(figsize=(8,5))

sns.barplot(
    x=models,
    y=accuracies,
    palette=["#6C8EBF", "#D9A66B", "#8E7DBE"]
)

plt.title(
    "Model Accuracy Comparison",
    fontsize=16,
    fontweight='bold'
)

plt.ylabel("Accuracy Score")
plt.xlabel("Models")

plt.ylim(0.7, 1.0)

plt.show()
```

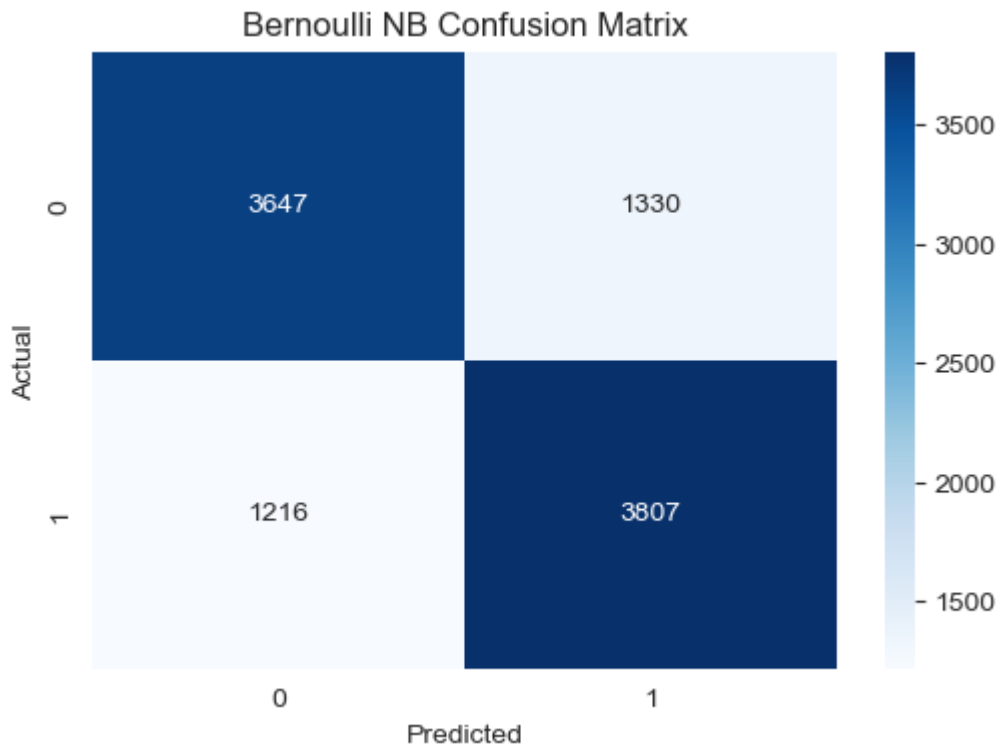


Insight

The comparison shows the relative performance of different machine learning models for sentiment classification. Models with higher accuracy demonstrate better capability in identifying positive and negative tweet sentiments.

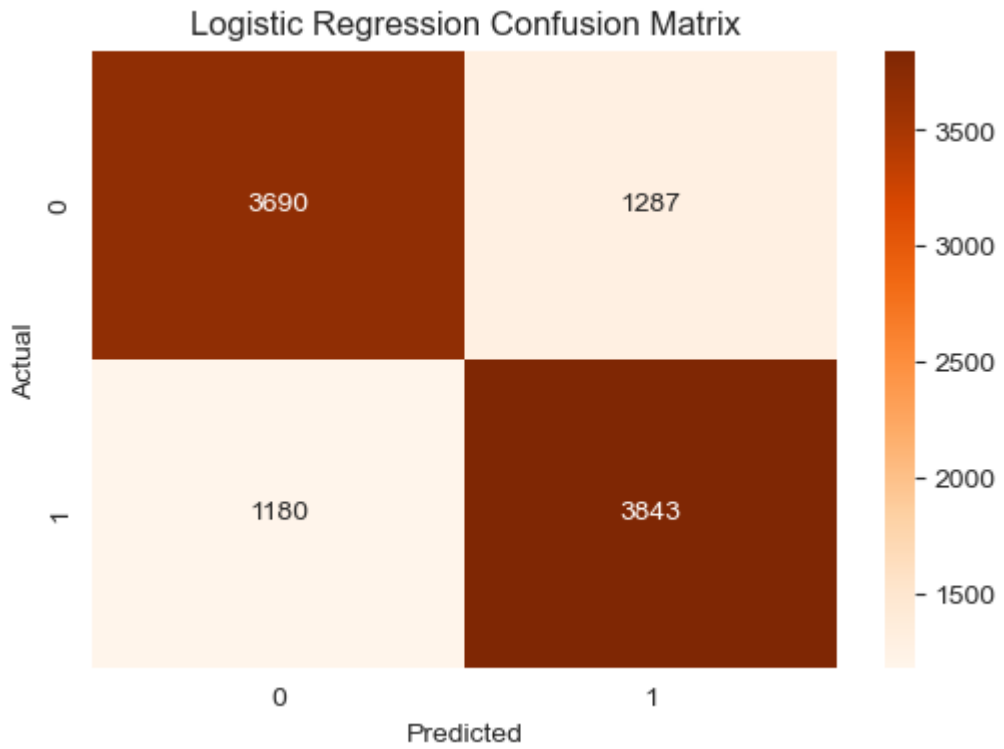
In [49]:
```
# STEP 16 – Confusion Matrix
# import matrics

from sklearn.metrics import confusion_matrix
```

In [50]:
```
# Bernoulli NB Confusion Matrix

bnb_cm = confusion_matrix(y_test, bnb_pred)

plt.figure(figsize=(6,4))

sns.heatmap(
    bnb_cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title("Bernoulli NB Confusion Matrix")

plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```



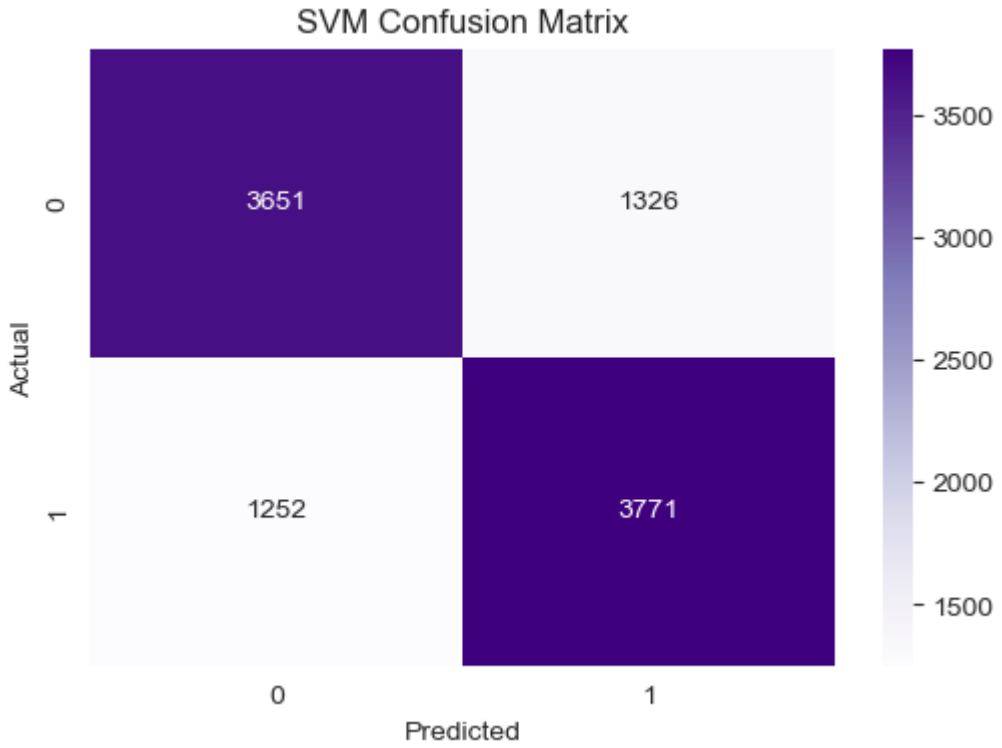
In [51]:
```
# Logistic Regression Confusion Matrix

lr_cm = confusion_matrix(y_test, lr_pred)

plt.figure(figsize=(6,4))

sns.heatmap(
    lr_cm,
    annot=True,
    fmt='d',
    cmap='Oranges'
)

plt.title("Logistic Regression Confusion Matrix")

plt.xlabel("Predicted")
plt.ylabel("Actual")
```

```
plt.show()
```



In [52]:
```
# SVM Confusion Matrix

svm_cm = confusion_matrix(y_test, svm_pred)

plt.figure(figsize=(6,4))

sns.heatmap(
    svm_cm,
    annot=True,
    fmt='d',
    cmap='Purples'
)

plt.title("SVM Confusion Matrix")
```

```
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```



Insight

Confusion matrices provide a detailed evaluation of model predictions by showing correctly and incorrectly classified tweets for both positive and negative sentiment classes.

ROC-AUC Curve Analysis

The ROC-AUC Curve evaluates the ability of machine learning models to distinguish between positive and negative sentiment classes. Higher AUC values indicate better classification performance.

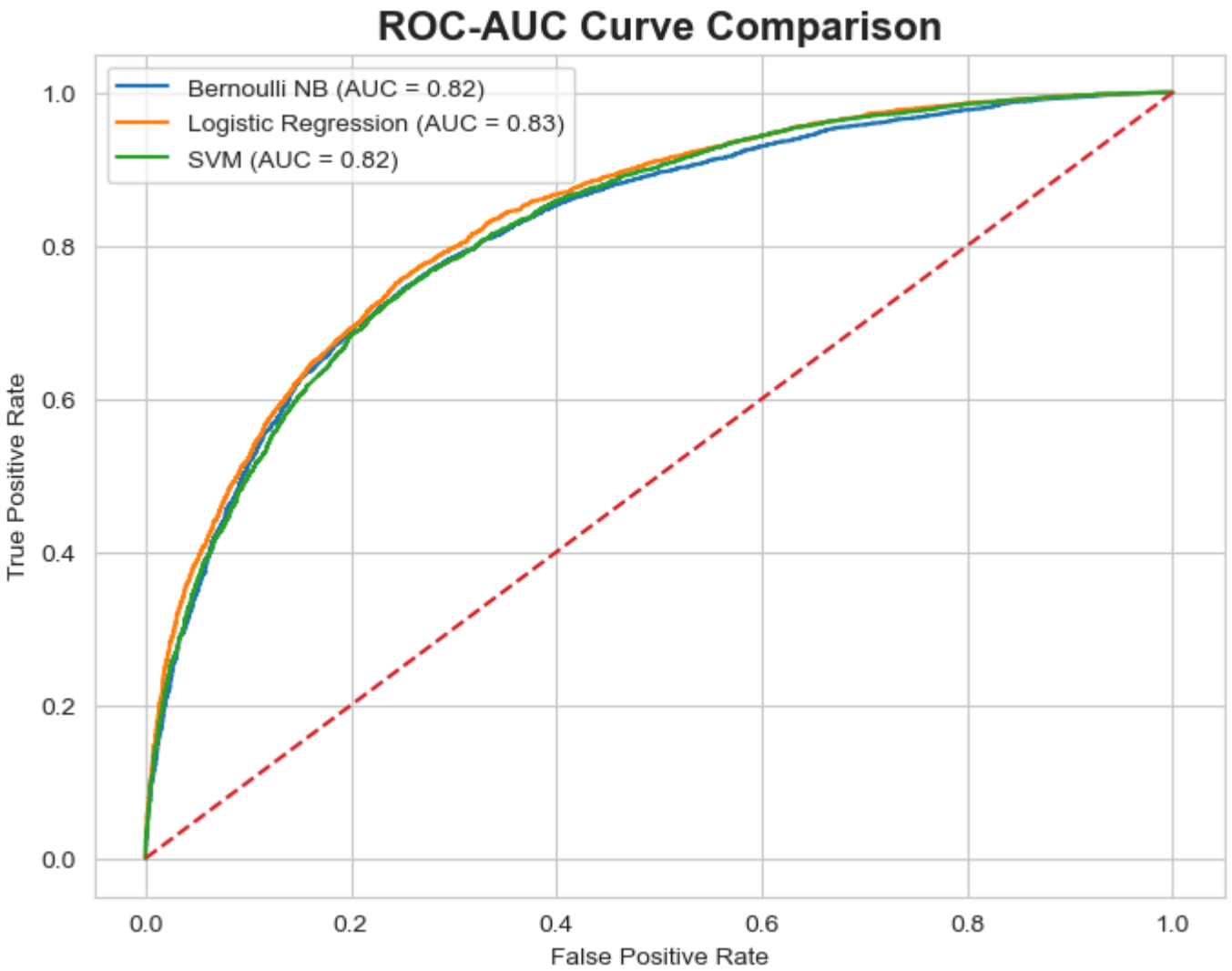
In [53]:
```
# STEP 20 – ROC-AUC Curve

#Import Libraries
from sklearn.metrics import roc_curve, auc
```

In [54]:
```
# Bernoulli NB ROC Curve

bnb_probs = bnb_model.predict_proba(X_test_tfidf)[:,1]

fpr_bnb, tpr_bnb, _ = roc_curve(y_test, bnb_probs)

roc_auc_bnb = auc(fpr_bnb, tpr_bnb)
```

In [55]:
```
# Logistic Regression ROC Curve

lr_probs = lr_model.predict_proba(X_test_tfidf)[:,1]

fpr_lr, tpr_lr, _ = roc_curve(y_test, lr_probs)

roc_auc_lr = auc(fpr_lr, tpr_lr)
```

In [56]:
```
# SVM ROC Curve

from sklearn.calibration import CalibratedClassifierCV

svm_calibrated = CalibratedClassifierCV(svm_model)

svm_calibrated.fit(X_train_tfidf, y_train)

svm_probs = svm_calibrated.predict_proba(X_test_tfidf)[:,1]

fpr_svm, tpr_svm, _ = roc_curve(y_test, svm_probs)

roc_auc_svm = auc(fpr_svm, tpr_svm)
```

In [57]:
```
# Plot ROC Curves

plt.figure(figsize=(8,6))

plt.plot(
    fpr_bnb,
    tpr_bnb,
    label=f'Bernoulli NB (AUC = {roc_auc_bnb:.2f})'
)

plt.plot(
    fpr_lr,
    tpr_lr,
    label=f'Logistic Regression (AUC = {roc_auc_lr:.2f})'
)

plt.plot(
    fpr_svm,
    tpr_svm,
    label=f'SVM (AUC = {roc_auc_svm:.2f})'
)

plt.plot([0,1], [0,1], linestyle='--')

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")

plt.title(
    "ROC-AUC Curve Comparison",
    fontsize=16,
    fontweight='bold'
)

plt.legend()

plt.show()
```



Insight

ROC-AUC analysis demonstrates the classification capability of each model across different threshold values. Models with higher AUC values provide stronger sentiment prediction performance.

In [58]:
```
results = pd.DataFrame({
    'Model': ['Bernoulli NB', 'Logistic Regression', 'SVM'],
    'Accuracy': [
        bnb_accuracy,
        lr_accuracy,
        svm_accuracy
    ]
})

results
```

Out[58]:

	Model	Accuracy
0	Bernoulli NB	0.7454
1	Logistic Regression	0.7533
2	SVM	0.7422

Business Impact

The developed sentiment analysis system can help organizations monitor customer opinions on social media platforms in real time. By automatically identifying negative sentiment trends, businesses can improve customer engagement, reputation management, and decision-making processes.

Final Model Selection

Among the evaluated models, Logistic Regression demonstrated strong classification performance with high accuracy and stable prediction capability on unseen tweet data.

The model effectively handled high-dimensional TF-IDF features while maintaining computational efficiency and interpretability. Due to its balanced performance and scalability, Logistic Regression was selected as the final model for this sentiment analysis task.

Conclusion

This project successfully implemented Twitter sentiment analysis using Natural Language Toolkit (NLTK), TF-IDF vectorization, and machine learning algorithms. Exploratory Data Analysis and text preprocessing techniques improved the quality of tweet data for classification tasks.

Among the evaluated models, Logistic Regression delivered strong sentiment prediction performance, making it suitable for real-world social media sentiment monitoring applications.

The developed solution can help businesses automatically identify customer opinions, detect negative sentiment trends, and support data-driven decision-making through social media analytics.